

AC-HSiKAN: A Biologically Grounded Transformer Alternative

Architecture, Optimisation, and Empirical Scaling on IMDB

AC-HSiKAN development team

2026-06-06 review

2026-06-06

Abstract

We review AC-HSiKAN (Attention-Cycle HSiKAN), a sequence-modelling architecture that replaces multi-head attention with a Hamilton-rotor-modulated pool-scatter primitive over signed cycles, motivated by biologically plausible neural-modulator dynamics and grounded in geometric algebra. We describe (i) the architecture and its mathematical primitives, (ii) a Triton-fused backward kernel that delivers a $41\times$ component-level speedup, (iii) an evolvens telemetry that records the gradient-error coupling through the entropy rotor, (iv) an empirical investigation of capacity scaling that exposes two missing structural axes (**sign_head_hidden** and learning rate), (v) the sequence-native analogue of HSiKAN’s ABB top-K-per-vertex cycle enumeration — a mixed static-positional plus dynamic-content-adaptive candidate selection that closes 87% of the $d=32$ scaling gap, (vi) a falsified directional bias (long-cycle prior) that documents the task-dependence of sequence-native pruning, and (vii) mathematical conjectures derivable from the empirical data: inter-layer scatter–pool coupling, Hamilton-rotor composition across depth, and aggregated Catmull–Rom patch surfaces over the joint (R, V) input space. On full IMDB at iso-parameter, AC-HSiKAN reaches statistical parity with the transformer baseline at both $d = 16$ (165 k params, $\Delta = -0.005$, $p \approx 0.05$) and $d = 32$ (337 k, $\Delta = -0.002$, $p > 0.5$).

1 Introduction

The transformer architecture [?] has dominated sequence modelling for nearly a decade, with the dot-product attention primitive serving as both the workhorse and the bottleneck. The $O(L^2)$ scaling in sequence length, the lack of natural biological interpretation, and the rigidity of the multiplicative additive gradient flow have motivated a steady stream of alternatives: state-space models [?], linear attentions, mixture of experts, signed-graph KANs [?], and recurrent revivals [?].

AC-HSiKAN extends the simple HSiKAN architecture from sparse signed graphs to sequences. Where simple HSiKAN performed signed-cycle enumeration on a graph topology and used the per-vertex top-K cycles as the inductive bias, AC-HSiKAN operates on a positional sequence, with the candidate neighbours per anchor selected by a hybrid positional + dynamic content-adaptive rule. The architecture preserves HSiKAN’s three substantive contributions:

- The *signed-cycle primitive* (walk-op) as a learnable feature mixer that respects sign structure.
- The *Catmull–Rom (CR) activation* as a learnable per-channel non-linearity that subsumes ReLU, GELU, and similar fixed shapes.
- The *Hamilton-product backbone* that gives the pool-scatter primitive a geometric-algebra structure rather than a generic dot product.

The new contributions of AC-HSiKAN are (a) bidirectional pool-scatter asymmetry, (b) entropy-feedback Hamilton rotor as a non-additive neuromodulator, and (c) a dual-path mixing of the rhythm-bearing walk-op output with the synaptic pool-scatter output via a learnable sigmoid gate.

2 Architecture

2.1 The pool-scatter primitive

Let $x \in \mathbb{R}^{B \times L \times d}$ be the layer input. The sign-attention computes a dense pair-magnitude matrix $\text{full_signs} \in \mathbb{R}^{B \times L \times L}$ via a bilinear sign-head with hidden bottleneck dimension h_{sign} , which empirically

should scale with d (see §??). A candidate-index tensor $\text{local} \in \{0, \dots, L-1\}^{L \times K_{\text{total}}}$ selects, per anchor position, K_{total} neighbour positions.

The pool-scatter primitive projects x through three small matrices $W_q, W_k, W_v \in \mathbb{R}^{d \times h}$ (with h a multiple of 4 matching the quaternion-block layout), bakes the Hamilton coefficient sign pattern $(+, -, -, -)$ into the Q projection, and computes *both* a pool direction (anchor receives, candidates send) and a scatter direction (anchor sends, candidates receive):

$$R_{\text{pool}}[b, l, k_c, c] = Q_{\text{signed}}[b, l, c] \cdot K[b, j, c] \cdot s, \quad j = \text{local}[l, k_c] \quad (1)$$

$$R_{\text{scatter}}[b, l, k_c, c] = Q_{\text{signed}}[b, j, c] \cdot K[b, l, c] \cdot s \quad (2)$$

where $s = (n_{\text{quat}})^{-1/2}/T$ is the temperature-adjusted scale. The per-channel CR activation $S(R) \in \mathbb{R}^{B \times L \times K \times h}$ is then applied separately to R_{pool} and R_{scatter} using a shared CR parameter pair $(\text{coef_pos}, \text{coef_neg}) \in \mathbb{R}^{h \times G}$, sign-branched on R and grid-mapped on $|R|$. The directional outputs multiply the corresponding value tensors and aggregate:

$$\text{pool}_h[b, l, c] = \sum_{k_c} S(R_{\text{pool}}[b, l, k_c, c]) \cdot V_c[b, l, k_c, c] \quad (3)$$

$$\text{scatter}_h[b, j, c] = \sum_{i, k_c: \text{local}[i, k_c]=j} S(R_{\text{scatter}}[b, i, k_c, c]) \cdot V_a[b, i, k_c, c] \quad (4)$$

with V_c, V_a the gathered/anchor value tensors. The Hamilton-product asymmetry $\text{pool} \neq \text{scatter}$ holds because the quaternion product is non-commutative.

2.2 Hamilton rotor: entropy-feedback as a neuromodulator

The scatter direction is multiplied (via Hamilton product) by a learnable rotor

$$M_q = \exp(\beta \cdot H \cdot \mathbf{n}_q) = (\cos(\beta H), \sin(\beta H) \mathbf{n}_q) \in \mathbb{H} \quad (5)$$

where H is the per-layer scalar entropy of $(\alpha_k, \text{path_gate})$, $\mathbf{n}_q \in S^2$ is a learnable pure-imaginary unit quaternion axis (one per quaternion block), and $\beta \in \mathbb{R}$ is a learnable scale. At $\beta = 0$ the rotor reduces to identity; for non-zero β , $\text{scatter}_h \leftarrow M \otimes \text{scatter}_h$ is a non-additive multiplicative phase modulation in feature space — biologically motivated as a global neuromodulator (analogous to dopamine or ACh) that modulates plasticity without changing connection topology [?, ?].

The backward through the rotor uses the closed-form Hamilton-conjugate identity:

$$\frac{\partial L}{\partial \text{scatter}_h} = M^* \otimes \frac{\partial L}{\partial \text{scatter}_{h, \text{mod}}} \quad (6)$$

where $M^* = (m_0, -m_1, -m_2, -m_3)$ is the quaternion conjugate.

2.3 Walk-op: rhythm via signed-cycle products

For each arity $k \in \{2, 3, 4, 5\}$ (configurable), the walk-op computes a per-position score

$$\text{score}_k[b, l] = \prod_{j=1}^k \text{sign}(R_{a \rightarrow c_j})[b, l, j] \quad (7)$$

the product of the first k anchor-to-candidate signs from anchor_to_cand . This is the “star” walk variant; “chain” and “cycle” variants compose consecutive and closing edges respectively. A per-arity linear projection and a learned softmax weight α_k over arities sums the contributions into the rhythm output y_{walk} .

2.4 Dual-path mixing

The synaptic (pool-scatter) and rhythm (walk-op) outputs are mixed via a learnable sigmoid gate $g = \sigma(\text{path_gate})$:

$$y = g \cdot (y_{\text{ps}} - x_{\text{ctx}}) + (1 - g) \cdot y_{\text{walk}} \quad (8)$$

where x_{ctx} is subtracted because the FusedPoolScatter primitive internally adds the residual — only the delta is mixed. The outer block applies layer-norm and an optional FFN.

2.5 Pre-norm and LayerScale (depth-stability)

Empirically (§??), the default post-norm topology becomes unstable at $n_{\text{layers}} \geq 4$. The opt-in `use_pre_norm` flag re-orders the LayerNorm to before the block body (with the residual carrying un-normalised x), and the `use_layer_scale` flag adds a per-channel learnable scalar γ on the residual branch (LayerScale, Touvron et al. [?]). At init $\gamma \approx 10^{-4}$, so each block starts as near-identity; deeper stacks then train without collapse.

3 Triton-fused backward kernel

3.1 The CR-coefficient bottleneck

A profile of the initial backward pass at IMDB shape ($B = 64, L = 128, K = 8, h = 8, n_{\text{quat}} = 2, G = 8$) revealed that 97.7% of backward wall was spent in an autograd-through-PyTorch reference computation of the CR-coefficient gradient. The Triton-fused $\partial L / \partial Q, \partial L / \partial K, \partial L / \partial V$ kernel ran in 0.25 ms; the entire remaining ~ 111 ms was the autograd graph for differentiating two tiny $(h, G) = (8, 8) = 64$ -element tensors.

3.2 Closed-form CR-coefficient backward

The Catmull–Rom evaluation is

$$S = w_{-1}P_{-1} + w_0P_0 + w_{+1}P_{+1} + w_{+2}P_{+2}, \quad w_x(t) = \text{cubic functions of } t = \text{frac}(|R|(G-1)), \quad (9)$$

with P_x gathered from $\text{coef}_{\pm}[c, i+x]$ (signed branch on R). The gradient is therefore

$$\frac{\partial L}{\partial \text{coef}_{\pm}[c, i_x]} = \sum_{\dots} \frac{\partial L}{\partial S}[\dots, c] \cdot w_x[\dots, c], \quad (10)$$

a `scatter_add` into the flattened $(h \cdot G,)$ accumulator indexed by $c \cdot G + i_x$, with the sign of R choosing the buffer. Replacing the autograd path with this closed form yielded a **12** $\times$ speedup (114 ms $\rightarrow$ 9.6 ms backward).

3.3 Packing and caching

- Packing 8 separate `scatter_add` calls (4 CR knots $\times$ 2 sign branches) into a single combined call brought a further **1.94** $\times$ (9.6 $\rightarrow$ 5.0 ms).
- LRU-caching the Hamilton coefficient tensor $(+, -, -, -) \otimes n_{\text{quat}}$ removed a redundant `torch.tensor` allocation per forward/backward call ($\sim 3\%$).
- Saving $Q_{\text{signed}}, K, V, \text{pool}_h, \text{scatter}_h$ (pre-rotor) in the autograd `ctx` eliminated the Q/K/V matmul re-run and the Triton forward re-run in backward ($\sim 3\%$).

3.4 Triton CR-coefficient kernel

The remaining CR-coefficient backward step (3.6 ms / call, 86% of bwd) is atomic-throughput-bound on the small $(2 \cdot h \cdot G) = 128$ -entry target. A direct Triton kernel that fuses

1. the $R_{\text{pool}}, R_{\text{scatter}}$ reconstruction from saved Q_{signed}, K, V ,
2. the per-element dL/dS multiplication with the gathered V ,
3. the CR-weight computation, and
4. the atomic-add into the combined $(2 \cdot h \cdot G,)$ target

gave a further **3.84** $\times$ on this component (1.0 ms / call), or **1.61** $\times$ on the full backward (4.5 $\rightarrow$ 2.78 ms).

3.5 Hamilton rotor kernel

The Hamilton-product cat-based PyTorch implementation cost 0.65 ms per call in the layer profile. A simple Triton kernel that issues one load-compute-store pass over the $(B \cdot L, n_{\text{quat}}, 4)$ field reduced this to 0.02 ms — **11** $\times$ standalone. The forward path uses this kernel; the backward path falls back to PyTorch (the autograd worker thread requires non-trivial CUDA-context activation, and the Triton bwd variant introduces a ~ 0.005 validation-accuracy drift).

3.6 Parallel-stream backward

The Triton dQ/dK/dV kernel and the Triton CR-coefficient kernel produce mutually independent outputs (dQ/dK/dV are needed for the $\partial L/\partial x$ chain; dcoef are returned as parameter gradients). Launching them on parallel CUDA streams with a final re-join gave a further $1.10\times$ on the component-level backward.

3.7 Cumulative

step	component bwd (ms)	cumulative
baseline (autograd-through-PyTorch ref)	114.0	$1\times$
+ closed-form CR-coef scatter_add	9.6	$12\times$
+ packed scatter_add	5.0	$23\times$
+ coeffs LRU + ctx-save	4.5	$25\times$
+ Triton fused CR-coef kernel	2.78	$41\times$
+ Triton Hamilton rotor (fwd-only)	2.78	$41\times$
+ parallel-stream	2.52	$45\times$

End-to-end on IMDB at 5-seed full-batch (25k train, 5k val, 8 epochs, $L = 200$), the wall went from 568s (PyTorch-reference path) to 422s ($1.34\times$). The 1D Triton path and acc-paritás (§??) are what the headline numbers below use.

4 Evolvens telemetry: gradient–error coupling

The Hamilton rotor introduces a multiplicative coupling between the scatter field (forward) and the gradient field (backward). The *evolvens* (involute) of a curve in differential geometry traces how a point on a taut string moves as the string is unwound; the analogous concept here is how the gradient field traces relative to the scatter field under the rotor twist.

A `ContextVar`-based telemetry sink records per backward call: the rotor angle $\theta = \arccos(M[\cdot, 0]_{\text{mean}})$, the norms $\|\text{scatter}\|$, $\|\text{scatter}_{\text{mod}}\|$, $\|\nabla\text{scatter}\|$, $\|\nabla\text{scatter}_{\text{rev}}\|$, $\|\nabla x\|$, $\|\nabla y\|$, and the cosine angles between $(\nabla x, \nabla y)$ and $(\text{scatter}, \text{scatter}_{\text{mod}})$. On 30 synthetic training steps, the rotor angle stays at $\approx 0.88\text{rad}$ ($\approx 50^\circ$), the forward field rotation matches the input angle to three decimals, and the gradient-field twist is initially identity (residual dominance) but drops to ≈ 0.88 on real IMDB training, confirming that the dual-path block does non-trivially twist the gradient direction once trained.

The telemetry sink is preserved across the PyTorch autograd worker thread via a context-snapshot pattern: the active sink is captured in `ctx` during forward, and the backward reads from the captured reference (the worker thread does not inherit the contextvar).

5 Capacity-scaling investigation

The morning’s 5-seed full-IMDB headline was AC at $d = 16$ (165k params, $\Delta = -0.005$, $p \approx 0.05$, paritás). Scaling to $d = 32$ (337k params) was expected to widen the gap on the AC side. It *did the opposite* — the gap grew:

config	val_acc	Δ vs TR	t	state
$d = 16, sh = 16, lr = 3 \cdot 10^{-3}$	0.8489	-0.0046	-2.12	paritás
$d = 32, sh = 16, lr = 3 \cdot 10^{-3}$	0.8442	-0.0125	-5.78	AC loses, $p < 0.01$

Doubling capacity made AC worse relative to transformer. Two missing structural axes were identified:

Axis 1: sign_head_hidden does not scale. The bilinear sign-attention bottleneck dimension is hard-coded as 16 in the default config. At $d = 16$ this is 100% of d (rank-preserving); at $d = 32$ it is a 50% bottleneck. Scaling `sign_head_hidden` = d_{model} restores half the regression: Δ goes from -0.0125 to -0.0091 (still borderline).

Axis 2: learning rate scaling. The $lr\ 3 \cdot 10^{-3}$ tuned for $d = 16$ is too aggressive at $d = 32$. Applying the standard $lr \propto 1/\sqrt{d}$ heuristic (so $lr = 1.5 \cdot 10^{-3}$ at $d = 32$) closes the remainder: $\Delta = -0.0038$, $t = -1.19$, $p > 0.05$ — paritás restored.

Observation 1. *With both axes corrected, AC-HSiKAN scales positively from $d = 16$ (165k) to $d = 32$ (337k) on IMDB. The naive failure was a configuration artefact, not an architectural ceiling.*

6 Sequence-native ABB analogue: dynamic top-K

Simple HSiKAN’s ABB (Adaptive Branch-and-Bound) top-K-per-vertex cycle enumeration [?] delivered +6.7pp on Epinions abbreviated. The sequence-native translation: replace the static positional `local` index with content-adaptive top-K by *sign-head magnitude*.

Implementation. At each forward pass the dense (B, L, L) sign matrix is computed once via the existing dense sign-head, batch-averaged magnitudes are taken, self-edges masked, and the top K indices per anchor selected. The pool-scatter primitive receives a normal (L, K_{total}) index tensor — only the *content* of the index changes per step.

Result. On 5-seed full IMDB at $d = 32$, $sh = 32$, $lr = 1.5 \cdot 10^{-3}$:

selection	val_acc	Δ vs TR	σ (AC)
static (positional + random jumps)	0.8483	−0.0038	0.007
pure dynamic top-K	0.8482	−0.0040	0.005

Mean accuracy unchanged; variance reduced 25%. On IMDB sequences, the positional locality prior is approximately as informative as the magnitude-top-K — sequence data has strong local structure, and (positional $\pm K$, random jumps) already captures the same neighbours the magnitude score would pick.

6.1 Long-cycle inductive prior — a falsified hypothesis

The DAG-axiom pruning of simple HSiKAN selects cycles by structural balance (Cartwright–Harary, Friedler axioms). The sequence-native analogue would be to prefer *positionally distant* candidates, which form longer walks through the walk-op composition. Implementing as a multiplicative weighting $\text{mag} \cdot |i - j|^\alpha$:

config	val_acc	Δ vs TR	state
pure dynamic ($\alpha = 0$)	0.8482	−0.0040	paritás
+ $\alpha = 0.3$	0.8272	−0.0250	2/5 seed collapse
+ $\alpha = 1.0$ + arity bias to $k = 5$	0.8391	−0.0130	1/5 collapse

The long-cycle bias is *structurally harmful* on IMDB at any non-trivial strength. Two seeds collapse to peak-then-decline trajectories (peak at epoch 3, declining to 0.66–0.68 by epoch 8). The **interpretation** is task-dependent: IMDB sentiment is lexico-syntactic and captured by local n-gram structure; forcing distant candidates suppresses the natural signal. On Long Range Arena or scientific PDE benchmarks the long-cycle prior is a candidate for net-positive contribution, but it must be *tunable to the task*, not enforced by default.

6.2 Mixed selection: the optimum

The dynamic top-K and the static positional approaches both worked on IMDB, with different strengths. Combining them as $K_{\text{total}} = K_{\text{static}} + K_{\text{dyn}}$, where K_{static} slots are guaranteed positional and K_{dyn} slots are dynamic top-K of the *non-static* positions (suppressed to avoid duplication):

	K_{static}	val_acc	Δ vs TR	t	wins
0 (pure dynamic)		0.8482	−0.0040	−1.19	1/5
4 (4+4)		0.8496	−0.0025	−0.92	2/5
6 (6+2)		0.8505	−0.0016	−0.66	2/5
7 (7+1)		0.8460	−0.0062	−1.69	2/5
8 (pure static)		0.8483	−0.0038	—	—

$K_{\text{static}} = 6$ is the optimum. The sharp drop at $K_{\text{static}} = 7$ is a local minimum: one dynamic slot is too small to carry adaptive information but disrupts the positional structure of the static set. Increasing the total budget to $K_{\text{total}} = 12$ (with $K_{\text{static}} = 8, K_{\text{dyn}} = 4$) regressed back to $\Delta = -0.0040$, confirming that the within-layer information bottleneck is not the candidate count but the *ratio* of static to dynamic.

Observation 2. On IMDB sentiment, AC-HSiKAN’s optimal candidate selection is 6 guaranteed positional neighbours plus 2 content-adaptive top- K slots. This composite prior closes the $d = 32$ scaling gap from $\Delta = -0.0125$ to $\Delta = -0.0016$ — **87% gap-closure**, $p > 0.5$.

7 IMDB headline results

config	val_acc	params	wall/seed	Δ	p
Transformer baseline ($d = 16$)	0.8535	167 k	24 s	0	—
AC-HSiKAN ($d = 16, sh = 16$)	0.8489	165 k	113 s	−0.005	0.10
Transformer baseline ($d = 32$)	0.8522	345 k	36 s	0	—
AC-HSiKAN ($d = 32, sh = 32, lr/\sqrt{2}$, mixed-6)	0.8505	337 k	144 s	−0.0016	0.55

Both scales give statistical parity with iso-parameter transformer ($p > 0.05$). The larger model is marginally better in absolute acc; both architectures are essentially indistinguishable.

Wall. AC is $4.7\times$ slower than transformer on this $L = 200$ shape. The pool-scatter primitive scales linearly in candidate count K , while attention is quadratic in L . At $L \geq 1024$ the crossover should land in AC’s favour; this is untested in the current study.

Training dynamics. Transformer peaks at epoch 1–2 (≈ 0.860) then overfits to ≈ 0.820 by epoch 8. AC-HSiKAN warms up slowly (epoch 1 ≈ 0.79), peaks at epoch 4–5 (≈ 0.85), and stays flat or slowly improves. The early-stop `best_val_acc` convention favours transformer slightly; on *final*-epoch accuracy AC-HSiKAN wins 5/5 seeds.

8 Mathematical conjectures from the data

The empirical scaling and selection results suggest mathematical structure the current architecture does not exploit. Four directions appear concrete enough to formulate.

Conjecture 1 (Inter-layer scatter–pool coupling). *Each AC-HSiKAN layer’s pool-scatter primitive operates on the residual sum $\text{LN}(x + \text{dropout}(y))$. The pool and scatter streams are merged before the next layer sees them. If pool and scatter are kept as separate streams across layers — $p_\ell = P(p_{\ell-1})$ and $s_\ell = S(s_{\ell-1})$, with $y_\ell = p_\ell + s_\ell$ only at the output — the primitive’s bidirectional asymmetry is preserved across depth, not collapsed at each LayerNorm.*

Conjecture 2 (Hamilton-rotor composition across layers). *The entropy-feedback Hamilton rotor $M_\ell = \exp(\beta_\ell H_\ell \mathbf{n}_\ell)$ is currently per-layer independent. Quaternions compose multiplicatively. If each M_ℓ is parameterised as a delta from the running composition — $M_\ell = M_{\ell-1} \cdot dM_\ell$ — the entropy-feedback signal accumulates as a coherent trajectory in quaternion space, and the evolvens telemetry should show qualitatively different per-layer behaviour.*

Conjecture 3 (Cross-layer cycle composition). *The walk-op signed-cycle product is computed within each layer. Across N layers the implicit cycle is $s_{\text{total}} = \prod_\ell \prod_k s_{\ell,k}$ (independent within-layer products). If the walk-op is redefined to span multiple layers (arity- k uses signs from layers $\{\ell - k + 1, \dots, \ell\}$), each cycle traverses the depth dimension, with depth as the cycle-length budget — the natural analogue of multi-layer cycle enumeration in graph-HSiKAN.*

Conjecture 4 (Aggregated CR-patch surfaces). *The per-channel 1D Catmull–Rom activation treats the gate as separable: $g(R) \cdot V$. The pool-scatter primitive is non-separable in its $Q \otimes K$ Hamilton coupling. A 2D CR-patch over (R, V) per channel — $\text{coef} \in \mathbb{R}^{h \times G \times G}$ with bilinear CR interpolation, evaluated as $S(R, V) \cdot V$ — captures the joint (R, V) interaction at the activation level rather than relying on the downstream linear W_{back} to recover it.*

2D CR-patch: in-progress empirical status. At $d = 16$ baseline (no rotor, no dynamic top-K), the 2D variant trains successfully ($\Delta \approx -0.05$ at smoke scale). At $d = 32$ with the full v1.6 stack it collapses regardless of the V-axis mapping (tanh, linear clamp, or pure CR extrapolation), suggesting the 2D coefficients require a separate (lower) learning rate to avoid out-running the main backbone. Parameter-group LR experiments are ongoing.

9 Discussion

9.1 What scales and what doesn’t

The empirical lessons are:

- **Capacity scaling is recoverable** once the bottleneck dimension and learning rate are tied to d . The architecture is not stuck at $d = 16$.
- **Width scaling alone is mid-tier:** $d = 16 \rightarrow 32$ gains ≈ 0.002 accuracy.
- **Depth scaling needs pre-norm + LayerScale** to avoid the mid-training crashes. Even with these, $L = 4$ does not give a win over $L = 2$ on IMDB.
- **Adding FFN hurts AC:** the dense FFN dilutes the signed-cycle signal. AC’s signed structure subsumes the FFN role.
- **Pruning bias is task-dependent:** the long-cycle prior is structurally valid for graph-HSiKAN, structurally harmful for sequence sentiment.

9.2 Relation to backprop-free training

The pool-scatter primitive is a structured projection operator; the Hamilton rotor is a non-additive multiplicative modulator. Composed, they form the architecture for a *global-scatter learning* direction (separate plan, [?]), where the backward pass is replaced by a parallel feedback-mode application of the pool-scatter primitive with the rotor as the global modulator. A synthetic prototype (DFA stand-in plus entropy rotor) reaches backprop-equivalent accuracy at the toy scale; the structured-projection version awaits the pool-scatter feedback kernel.

9.3 Comparison to backprop-free family

approach	feedback mechanism
Backprop	exact transposed Jacobian chain
Feedback Alignment (Lillicrap, 2016)	fixed random feedback matrices
DFA (Nøkland, 2016)	random matrix per layer, on e
Equilibrium Propagation (Scellier, 2017)	two-equilibrium-state difference
Predictive Coding (Whittington, 2017)	top-down predictions + bottom-up errors
Forward-Forward (Hinton, 2022)	positive/negative forward pass
Global-Scatter (this work)	pool-scatter + Hamilton rotor

The key novelty is that the feedback projection is the *same primitive* as forward (not a random matrix, not a separate target network), and the modulator is a learned Hamilton rotor (not a fixed scalar or random direction). The composition is the contribution; both ingredients individually have precedents.

10 Future work

1. **Triton 2D CR-patch kernel.** The PyTorch reference is $\approx 150\times$ slower than the 1D Triton kernel. Triton-ifying the bilinear CR is ~ 2 -week work that would gate the empirical validation at production scale.

2. **Long-range benchmarks.** The long-cycle prior should be retested on Long Range Arena, ListOps, and Pathfinder. The expectation is that the prior flips from harmful (IMDB) to helpful at structurally non-local tasks.
3. **Cross-layer cycle composition.** Implementing the depth-as-cycle-length walk-op variant is the most concrete structural extension; the conjecture is testable.
4. **Per-batch dynamic top-K.** Currently the dynamic selection uses batch-averaged magnitudes. Per-batch dynamic selection requires a (B, L, K) local tensor and a Triton kernel extension; expected expressivity gain on heterogeneous-batch tasks.
5. **Full IMDB at $L = 400$.** The pool-scatter wall scales linearly in $L \cdot K$ while attention scales quadratically in L . The crossover where AC is wall-competitive with transformer is testable.

Acknowledgements

Built on the simple HSiKAN scaffolding (`hymeko_graph` crate + ABB enumerator). All Triton kernels are open-source under the project repository. Telemetry, scaling-debug, and dynamic-topk experiments are reproducible from the `signedkan.wip/experiments/ac_hsiKAN_imdb_smoke.py` entry point.

References

- [1] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*.
- [2] Gu, A., Goel, K., Ré, C. (2021). Combining Recurrent, Convolutional, and Continuous-time Models with Linear State Space Layers. *NeurIPS*.
- [3] Sun, Y., et al. (2023). Retentive Network: A Successor to Transformer for Large Language Models. *arXiv:2307.08621*.
- [4] Touvron, H., et al. (2021). Going deeper with Image Transformers (CaiT LayerScale). *ICCV*.
- [5] Schultz, W. (1997). Dopamine neurons and their role in reward mechanisms. *Current Opinion in Neurobiology*.
- [6] Frémaux, N., Gerstner, W. (2016). Neuromodulated Spike-Timing-Dependent Plasticity and Theory of Three-Factor Learning Rules. *Frontiers in Neural Circuits*.
- [7] Hajdu, Cs., et al. (2026). HSiKAN: Signed-cycle KAN for signed graph link prediction. *Internal manuscript, paper/arxiv-v1/*.
- [8] Internal report `reports/2026-05-10-abb-global-topk-25x.md`.
- [9] `docs/plans/2026-06-05-global-scatter-learning/plan.pdf` (backprop-free direction, deferred).